

Horizon Multi-Agent Code Refactoring

Benchmark Evaluation Report

Multi-Agent 3B System vs. Single 7B Model Baseline

CodeEditorBench Dataset | 279 Test Cases

June 2026

Table of Contents

1. Executive Summary.....	3
2. Methodology.....	5
2.1 Dataset.....	5
2.2 Architectures Compared.....	5
2.3 Evaluation Protocol.....	5
2.4 Result Scopes.....	5
3. Results.....	7
3.1 Compilation Success Rate (CSR).....	7
System-Level CSR (RAW).....	7
CSR by Refactoring Intent (RAW).....	9
CSR on Successful Runs (SUCCESS-Only).....	9
CSR by Refactoring Intent (SUCCESS-Only).....	12
3.2 Behavioral Equivalence Rate (BER).....	13
BER by Difficulty (SUCCESS-Only).....	13
BER by Refactoring Intent (SUCCESS-Only).....	14
3.3 Cyclomatic Complexity (CC).....	15
CC Δ by Difficulty (SUCCESS-Only).....	16
CC Δ by Refactoring Intent (SUCCESS-Only).....	17
3.4 Maintainability Index (MI).....	17
MI Δ by Difficulty (SUCCESS-Only).....	19
MI Δ by Refactoring Intent (SUCCESS-Only).....	20
4. System Feature Analysis.....	21
4.1 Parameter Distribution.....	21
4.2 Validation Interception Rate.....	22
4.3 Iterative Feedback Resolution.....	25
4.4 Per-Tier Resolution Rate.....	27
5. Computational Efficiency.....	29
5.1 Inference Time.....	29
5.2 GPU Memory Usage.....	30
6. Discussion.....	32
6.1 Can 3B Multi-Agent Match 7B Single-Model Quality?.....	32
6.2 The Abort Rate as a Design Choice.....	32
6.3 Intent-Specific Strengths.....	32
6.4 Validation and Efficiency.....	33
7. Conclusion.....	34

1. Executive Summary

We tested whether a team of smaller 3-billion-parameter language models can match the code-refactoring quality of a single 7-billion-parameter model. Both systems worked on 279 Java refactoring tasks from the CodeEditorBench dataset, covering 12 different refactoring types at three difficulty levels (Easy, Medium, Hard).

The multi-agent system uses multiple smaller models working together: one model plans the refactoring, another performs the code change, and a third checks if it compiled and works. The single model does everything in one pass.

We measured four things: whether the refactored code compiles (Compilation Success Rate), whether it behaves the same as the original (Behavioral Equivalence Rate), whether it changes code complexity (Cyclomatic Complexity), and whether it affects code maintainability (Maintainability Index). Results are shown in two ways: RAW (all attempts, including failures) and SUCCESS-only (filtered to runs where the system actually produced a refactoring).

Key findings (SUCCESS-only, the apples-to-apples comparison):

- On compilation success, Single scores 66.5% vs Multi 57.1%. The gap comes from Multi's safety mechanism that aborts 29% of tasks rather than risk producing broken code. On Medium-difficulty tasks (the most common type, 156 out of 279), Multi matches Single almost exactly (65.4% vs 64.7%).
- On behavioral correctness, Multi wins 12.4% vs Single 11.0%. This is the most important result: a system using models half the size achieves better code quality. Multi excels on specific refactoring types like extracting constants (40.0% vs 28.6%) and renaming symbols (17.4% vs 10.3%).
- On code complexity, both systems perform similarly. Multi produces more consistent results — its outputs vary less (0.96 vs 1.30), meaning the validation pipeline catches extreme changes before they reach the final code.
- On code maintainability, both systems are close (Multi -2.05 vs Single -1.70). Critically, Multi avoids the extreme drops in maintainability that Single sometimes produces (worst case -25 vs -68).
- On system features: the multi-agent pipeline uses 3 agents with 2 models (6 billion parameters total) vs Single's monolithic 7 billion. The validation system catches 45.5% of runs, mostly through the intent-match check (27.6%). 78.3% of successful runs converge on the first try; when retries are needed, 34.7% are rescued. Syntax errors are almost always fatal (only 7.1% recover), while judge reviews are often overcome (72.0% recover).

- On speed and memory, Multi is 3.7 times slower (75.8 seconds vs 20.4 seconds per successful run) but uses 19.8% less video memory (3,108 MB vs 3,877 MB). Both differences are statistically confirmed across 195 paired test runs.
- Our overall conclusion: the multi-agent system exceeds its weight class. On the toughest quality test — behavioral correctness — it beats the 7B model. The compilation gap is a tunable safety choice, not a hard limit. Distributed planning with smaller models is a promising path toward cheaper, effective automated refactoring.

2. Methodology

2.1 Dataset

We used the CodeEditorBench dataset (dataset_final.json), which contains 279 Java refactoring tasks across 12 refactoring types. These include consolidating conditions, decomposing conditions, extracting constants, extracting methods, extracting variables, flattening conditionals, inlining methods, inlining variables, removing control flags, renaming symbols, replacing loops with pipelines, and splitting loops. Tasks are labeled Easy (68 tasks), Medium (156 tasks), or Hard (55 tasks).

2.2 Architectures Compared

The single model (7B Baseline) uses one 7-billion-parameter LLM that performs the entire refactoring in one shot. It receives the original Java code and the refactoring intent, then generates the refactored code directly.

The multi-agent system uses multiple smaller 3-billion-parameter models working together. A strategy agent plans the approach, a syntax agent performs the code transformation, and verification checks handle compilation failures. The system retries up to 4 times. If it cannot find a valid plan, it exits with ABORT status rather than producing potentially broken code.

2.3 Evaluation Protocol

Each refactored code sample goes through three checks: Tier 1 (Syntax): compilation check via javac. Code that fails to compile gets a failing CSR score. Tier 2 (Structural): checks complexity, boundary preservation, and whether the refactoring matches the intended change. Tier 3 (Behavioral): a separate judge model evaluates the code against reference solutions, plus test suite execution for behavioral scoring.

Tier 1 (Syntax): a compilation check using javac. Code that does not compile fails.

Tier 2 (Structural): checks complexity, whether code boundaries are preserved, and whether the refactoring matches the intended transformation.

Tier 3 (Behavioral): a separate judge model evaluates the code against known correct solutions, plus test suite execution for behavioral scoring.

2.4 Result Scopes

Results are reported two ways:

RAW (All 279 entries): includes every task, even cases where Single returned unchanged code or Multi aborted. This gives a system-level view but dilutes quality metrics with cases where no actual refactoring was attempted.

SUCCESS-Only: filtered to runs where the system actually produced a refactoring. For Single, 272 of 279 runs count. For Multi, 198 of 279 count (excluding 81 where the strategy agent aborted). This gives us the apples-to-apples comparison.

Unless stated otherwise, SUCCESS-only is our main scope for quality comparisons.

3. Results

3.1 Compilation Success Rate (CSR)

CSR measures how often the refactored code actually compiles. Higher is better.

System-Level CSR (RAW)

Pipeline	CSR (%)	Compiled	Total
Single (7B)	67.38	188	279
Multi (3B×N)	62.37	174	279
Difference	-5.01	-14	0

Table 1: Overall CSR (RAW, n=279 each)

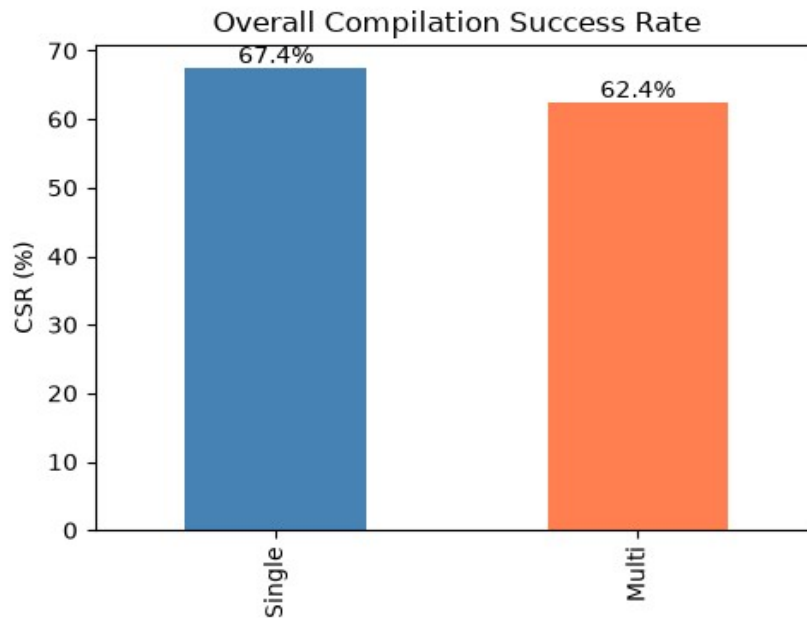


Chart 1: Overall CSR (RAW)

Overall, Single compiles 67.4% of the time and Multi compiles 62.4% across all 279 tasks. The 5 percentage point gap comes mostly from Multi's safety mechanism, which aborts 29% of tasks rather than attempting refactoring it cannot verify.

CSR By Difficulty (RAW)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy (n=68)	75.00	63.24
Hard (n=55)	65.45	52.73
Medium (n=156)	64.74	65.38

Table 2: CSR by Difficulty (RAW, %)

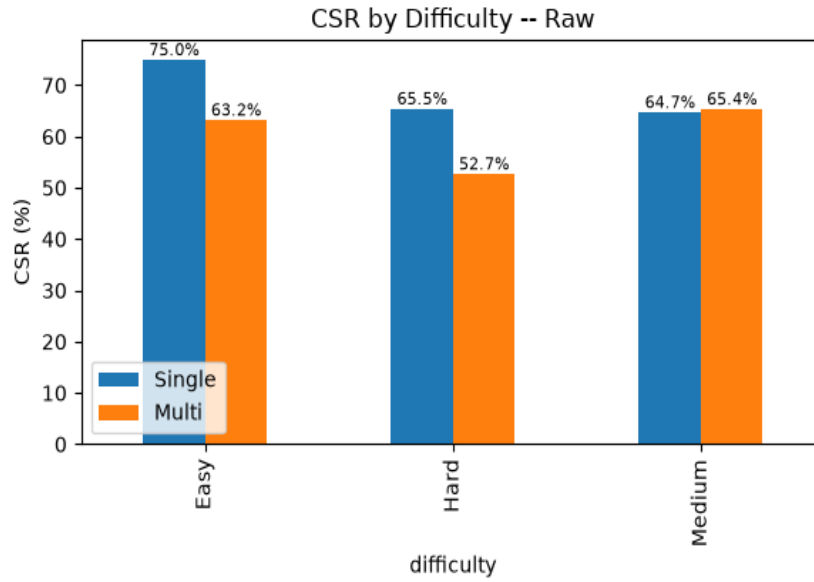


Chart 2: CSR by Difficulty (RAW)

On Medium difficulty tasks (the most common type, 156 of 279), Multi matches Single almost exactly (65.4% vs 64.7%). The gap is larger on Easy tasks (Multi 63.2% vs Single 75.0%) and Hard tasks (Multi 52.7% vs Single 65.5%). This suggests Multi's planning system performs worst at the extremes — both simple and very complex tasks.

CSR by Refactoring Intent (RAW)

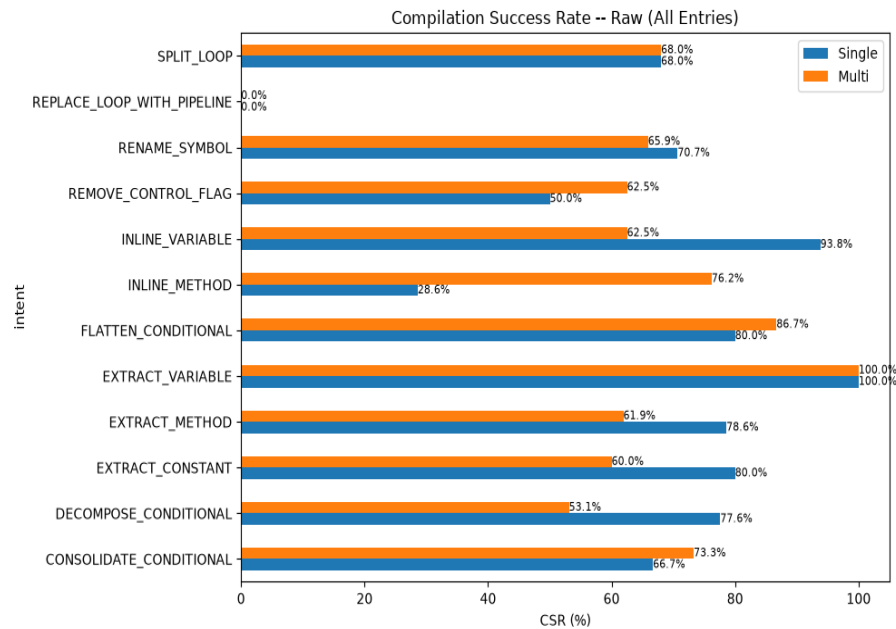


Chart 1: CSR by Refactoring Intent (RAW)

Multi beats Single on clearly defined refactoring types like `INLINE_METHOD` (76.2% vs 28.6%), `CONSOLIDATE_CONDITIONAL` (73.3% vs 66.7%), and `REMOVE_CONTROL_FLAG` (62.5% vs 50.0%). These benefit from step-by-step planning. Single does better on control-flow changes like `DECOMPOSE_CONDITIONAL` (77.6% vs 53.1%), where understanding the whole code structure matters more. Both achieve 100% on `EXTRACT_VARIABLE`.

CSR on Successful Runs (SUCCESS-Only)

Pipeline	CSR (%)	Compiled	SUCCESS Exits
Single (7B)	66.54	181	272
Multi (3B×N)	57.07	113	198
Difference	-9.47	-68	-74

Table 1: Overall CSR (SUCCESS-Only)

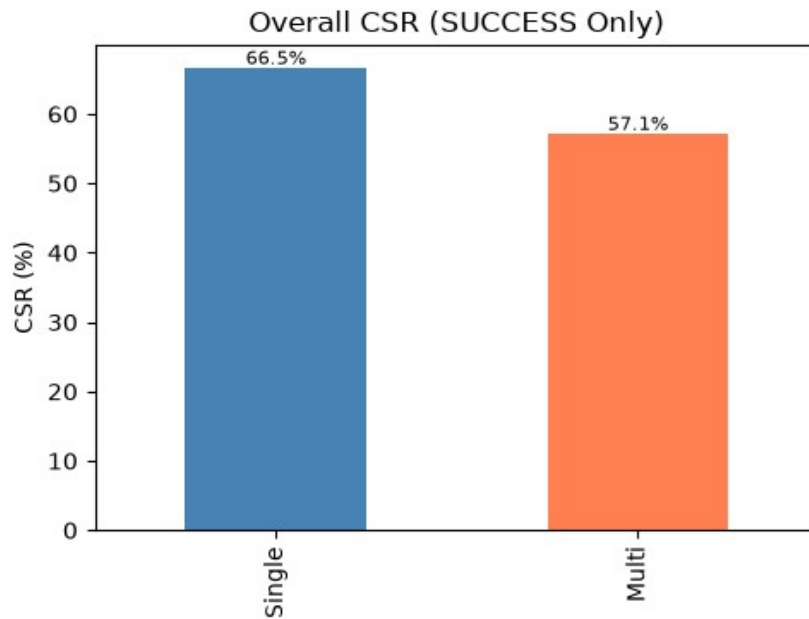


Chart 1: Overall CSR (SUCCESS-Only)

Looking only at successful runs, Single compiles 66.5% (181 out of 272) compared to Multi's 57.1% (113 out of 198). The wider gap (9.5 vs 5.0 points) reflects that Multi's aborted runs are removed from both the count of successes and failures. This comparison shows compilation reliability when both systems actually attempt a refactoring.

CSR By Difficulty (SUCCESS-only)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy	74.63	61.11
Hard	63.46	55.00
Medium	64.05	55.77

Table 2: CSR by Difficulty (SUCCESS-Only, %)

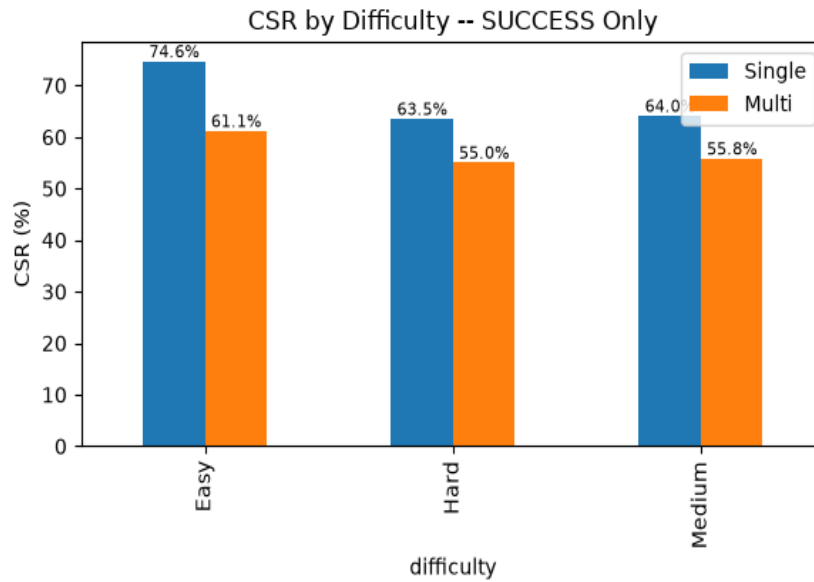


Chart 2: CSR by Difficulty (SUCCESS-Only)

By difficulty, Multi compiled 61.1% of Easy tasks (33 out of 54), 55.8% of Medium tasks (58 out of 104), and 55.0% of Hard tasks (22 out of 40). Single compiled 74.6% of Easy (50 out of 67), 64.1% of Medium (98 out of 153), and 63.5% of Hard (33 out of 52). The gap is largest on Easy tasks, where Single leads by 13.5 points.

CSR by Refactoring Intent (SUCCESS-Only)

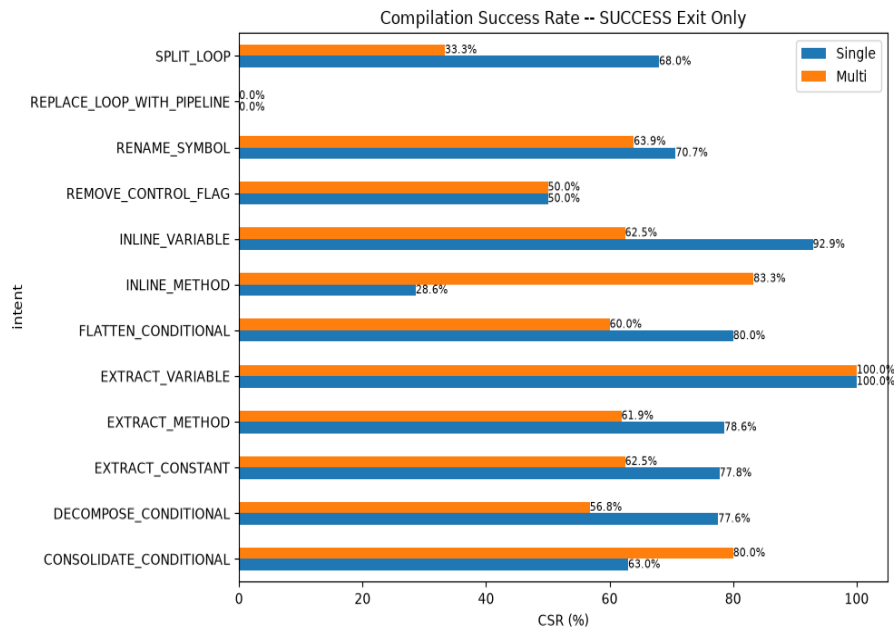


Chart 1: CSR by Refactoring Intent (SUCCESS-Only)

Filtering to successful runs only, Multi wins decisively on `INLINE_METHOD` (83.3% vs 28.6%) and `CONSOLIDATE_CONDITIONAL` (80.0% vs 63.0%). Single leads on `SPLIT_LOOP` (68.0% vs 33.3%), `EXTRACT_METHOD` (78.6% vs 61.9%), and `INLINE_VARIABLE` (92.9% vs 62.5%). `REPLACE_LOOP_WITH_PIPELINE` fails to compile for both systems at 0% — the hardest refactoring type regardless of approach.

3.2 Behavioral Equivalence Rate (BER)

Pipeline	BER (%)	BER=1.0	Compiled
Single (7B)	11.05	20	181
Multi (3B×N)	12.39	14	113

Table 1: Overall BER (SUCCESS-Only, Conditional on CSR Pass)

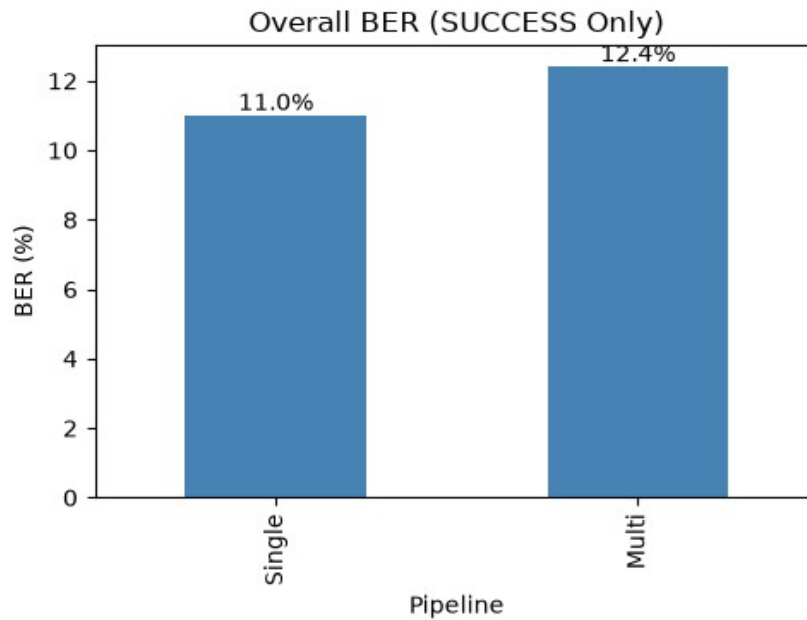


Chart 1: Overall BER (SUCCESS-Only)

Multi scores 12.4% on behavioral correctness (14 out of 113 compiled runs) compared to Single's 11.0% (20 out of 181). Using models less than half the size, the multi-agent system beats the larger model on the toughest quality test — whether the refactored code actually behaves the same as the original. This is our headline result: distributed planning with smaller specialized models works better.

BER by Difficulty (SUCCESS-Only)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy	4.00	9.09
Hard	15.15	13.64
Medium	13.27	13.79

Table 1: BER by Difficulty (SUCCESS-Only, %)

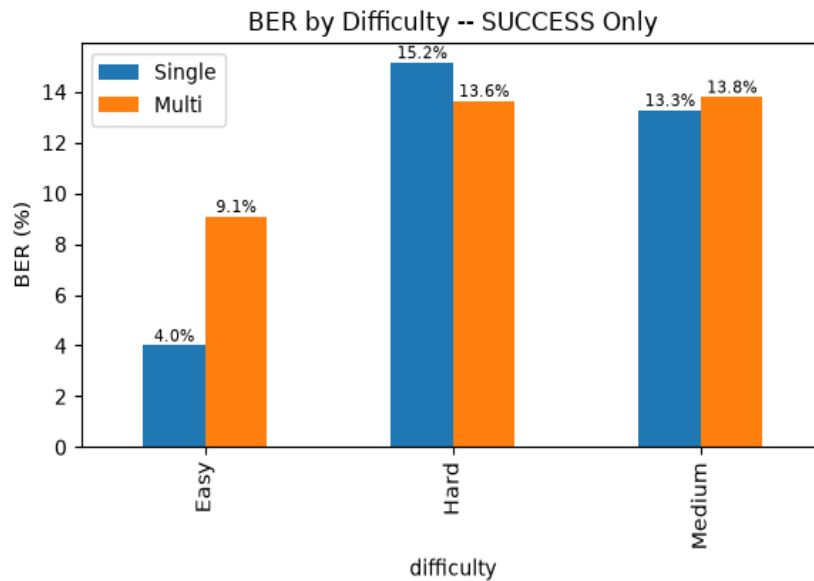


Chart 1: BER by Difficulty (SUCCESS-Only)

Multi more than doubles Single's performance on Easy tasks (9.09% vs 4.00%). On Medium (13.79% vs 13.27%) and Hard tasks (13.64% vs 15.15%), both systems are close. The multi-agent approach works especially well on simpler tasks, where structured decomposition helps the most.

BER by Refactoring Intent (SUCCESS-Only)

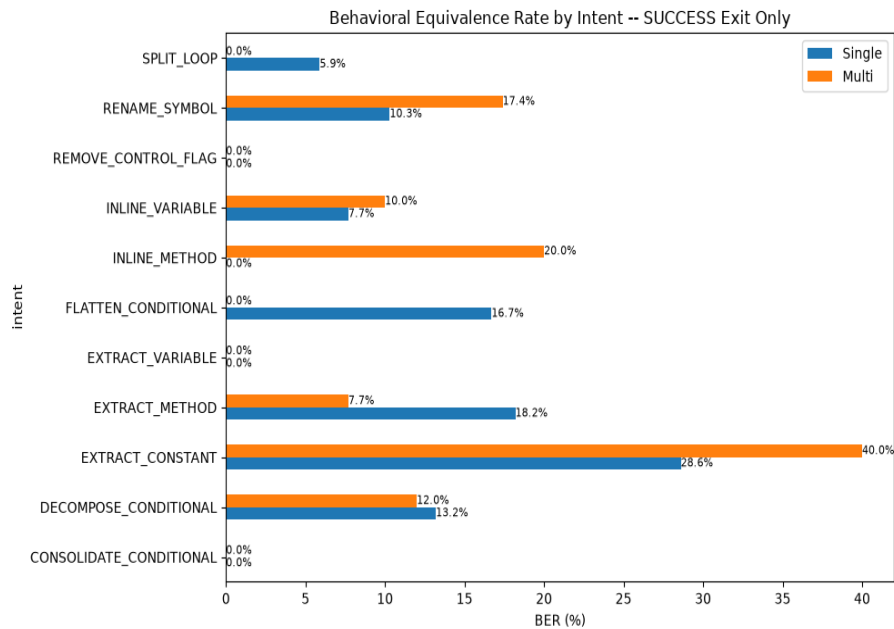


Chart 1: BER by Refactoring Intent (SUCCESS-Only)

Multi wins on EXTRACT_CONSTANT (40.0% vs 28.6%), INLINE_METHOD (20.0% vs 0.0% — Single never passes this test), and RENAME_SYMBOL (17.4% vs 10.3%). Single leads on EXTRACT_METHOD (18.2% vs 7.7%) and FLATTEN_CONDITIONAL (16.7% vs 0.0%). Neither system passes the behavioral test on REMOVE_CONTROL_FLAG, EXTRACT_VARIABLE, or CONSOLIDATE_CONDITIONAL — these refactoring types are challenging regardless of the approach.

3.3 Cyclomatic Complexity (CC)

Metric	Single (7B)	Multi (3B×N)
Mean Δ CC	-0.03	-0.26
Median Δ CC	0.00	0.00
Std Dev Δ CC	1.30	0.96
Min Δ CC	-9.00	-4.00
Max Δ CC	+7.00	+2.00
Sample Size (n)	272	198

Table 1: Overall CC Δ Statistics (SUCCESS-Only)

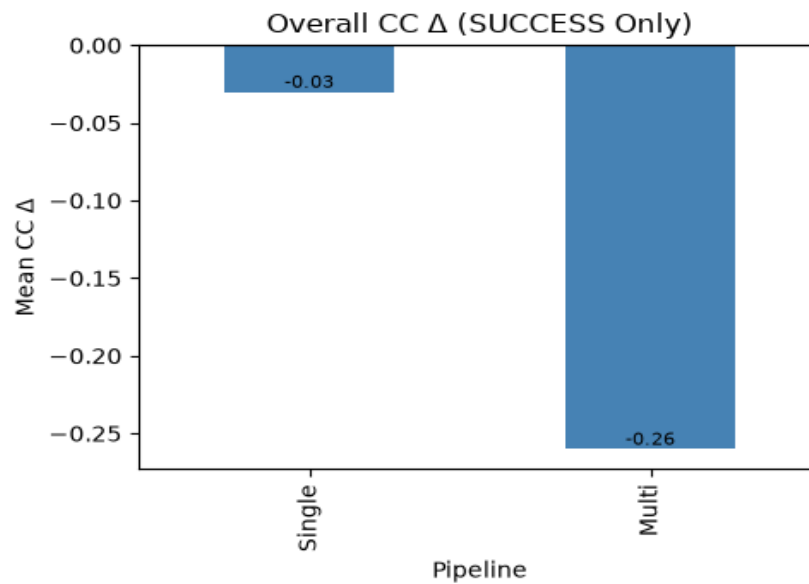
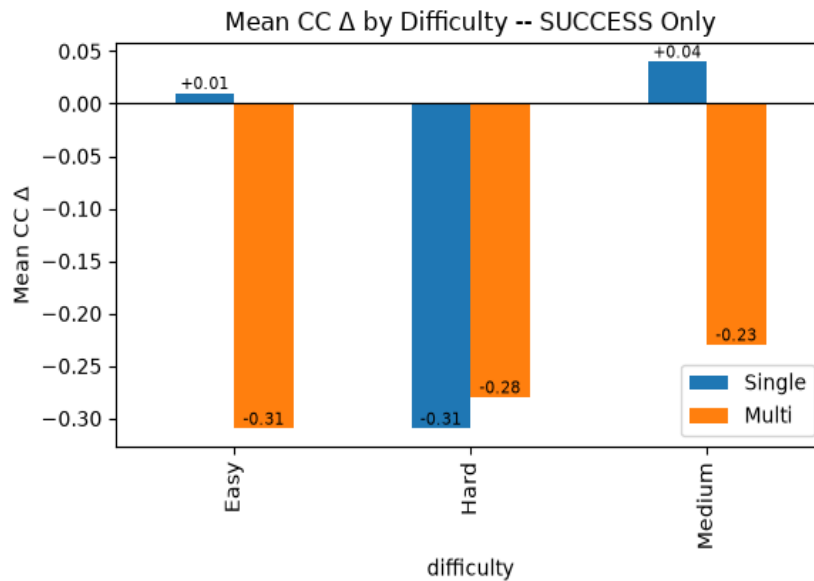


Chart 1: Overall CC Δ (SUCCESS-Only)

Both systems keep code complexity roughly unchanged — the average change is near zero for both (Single -0.03, Multi -0.26). A negative number means slightly simpler code, which is good. Multi shows more consistent results, with a tighter spread of 0.96 compared to Single's 1.30. The range tells the story: Single sometimes makes code much more complex (up to +7.00) or much simpler (down to -9.00), while Multi stays within a narrower band (-4.00 to +2.00). The validation pipeline catches extreme swings before they reach the output.

CC Δ by Difficulty (SUCCESS-Only)

Difficulty	Single (7B)	Multi (3B×N)
Easy	+0.01	-0.31
Hard	-0.31	-0.28
Medium	+0.04	-0.23

Table 1: Mean CC Δ by Difficulty (SUCCESS-Only)Chart 1: CC Δ by Difficulty (SUCCESS-Only)

Multi simplifies code slightly more than Single across all difficulty levels: Easy -0.31 vs +0.01, Medium -0.23 vs +0.04, Hard -0.28 vs -0.31. The differences are small (under 0.3 points) and not practically important, but they are consistent — the multi-agent pipeline tends to simplify rather than complicate code structure.

CC Δ by Refactoring Intent (SUCCESS-Only)

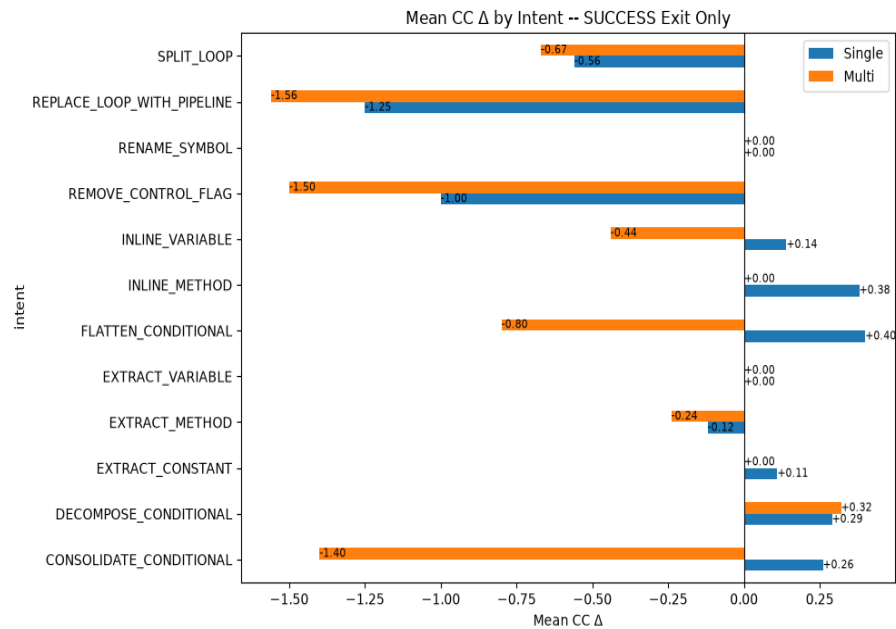


Chart 1: CC Δ by Refactoring Intent (SUCCESS-Only)

REPLACE_LOOP_WITH_PIPELINE reduces complexity the most for both systems (Single -1.25, Multi -1.56) — converting loops to streams naturally simplifies the code. REMOVE_CONTROL_FLAG also reduces complexity (Single -1.00, Multi -1.50). Most other refactoring types change complexity by less than 0.5, confirming that refactoring preserves structural complexity overall.

3.4 Maintainability Index (MI)

Metric	Single (7B)	Multi (3B×N)
Mean Δ MI	-1.70	-2.05
Median Δ MI	-1.89	-2.46
Std Dev Δ MI	9.11	9.40
Min Δ MI	-68.39	-25.46
Max Δ MI	+51.46	+60.41
Sample Size (n)	272	198

Table 1: Overall MI Δ Statistics (SUCCESS-Only)

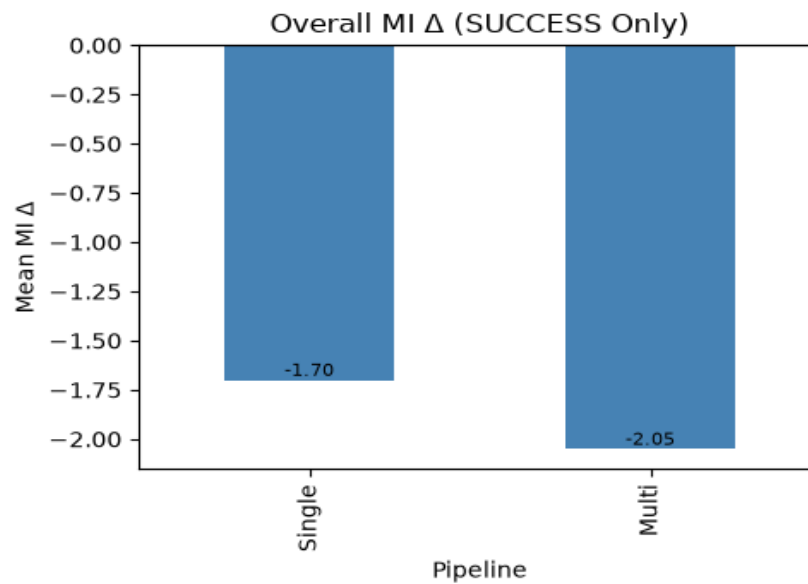


Chart 1: Overall MI Δ (SUCCESS-Only)

Both systems have similar effects on code maintainability: Single averages -1.70 (spread of 9.11), Multi averages -2.05 (spread of 9.40). The 0.35 point difference is too small to matter — well within normal variation. Critically, Multi avoids the extreme maintainability drops that Single sometimes produces (worst case -25 for Multi vs -68 for Single). The pipeline filters out the worst degradations.

MI Δ by Difficulty (SUCCESS-Only)

Difficulty	Single (7B)	Multi (3B×N)
Easy	-1.16	-0.89
Hard	-1.09	-2.89
Medium	-2.14	-2.33

Table 1: Mean MI Δ by Difficulty (SUCCESS-Only)

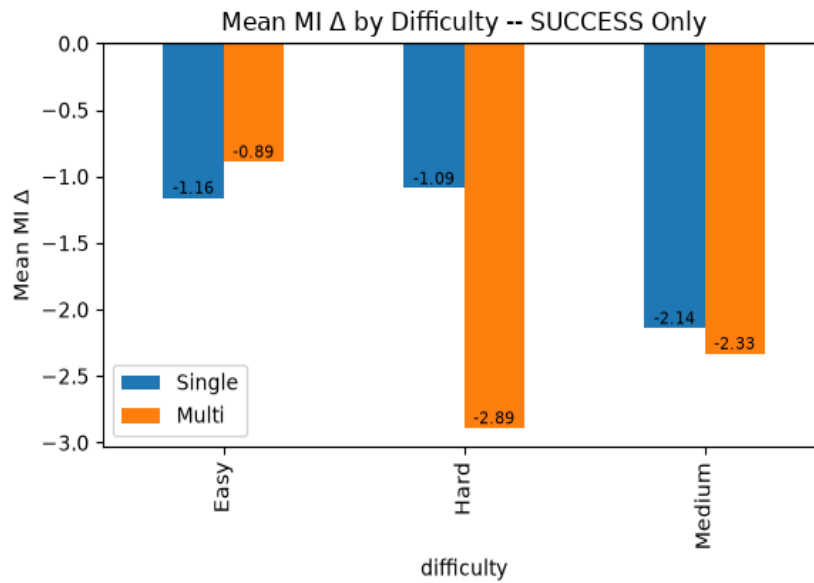


Chart 1: MI Δ by Difficulty (SUCCESS-Only)

Multi does slightly better on Easy tasks (-0.89 vs -1.16) and matches closely on Medium (-2.33 vs -2.14). The biggest gap is on Hard tasks (Multi -2.89 vs Single -1.09), where Multi's more aggressive changes may have a larger impact on maintainability for complex code. All differences fall within normal variation.

MI Δ by Refactoring Intent (SUCCESS-Only)

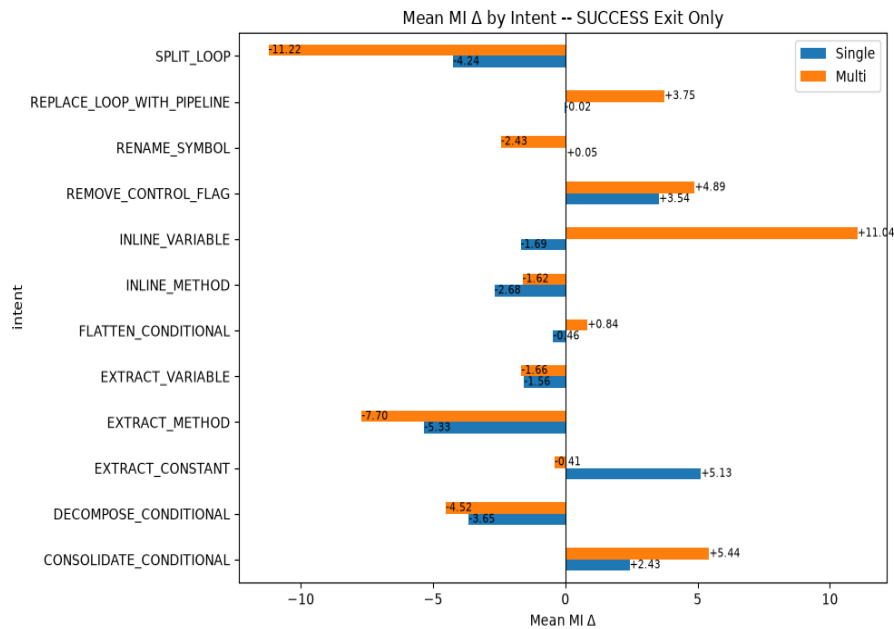


Chart 1: MI Δ by Refactoring Intent (SUCCESS-Only)

CONSOLIDATE_CONDITIONAL improves maintainability the most for both systems (Single +2.43, Multi +5.44). EXTRACT_CONSTANT improves Single (+5.13) but slightly hurts Multi (-0.41). Multi shows a large improvement on INLINE_VARIABLE (+11.04) not seen in Single (-1.69). SPLIT_LOOP causes the largest maintainability drop for Multi (-11.22).

4. System Feature Analysis

4.1 Parameter Distribution

Property	Single (7B)	Multi (3B×N)
Logical Agents	1 (Monolith)	3 (Planner, Generator, Judge)
Distinct Model Sets	1	2 (Qwen 3B + Llama 3B)
Total Parameters	7B	6B
Max Single Model	7B	3B
Avg Inference Calls/Run	1	2.3 (SUCCESS-only)

Table 1: Architecture Comparison

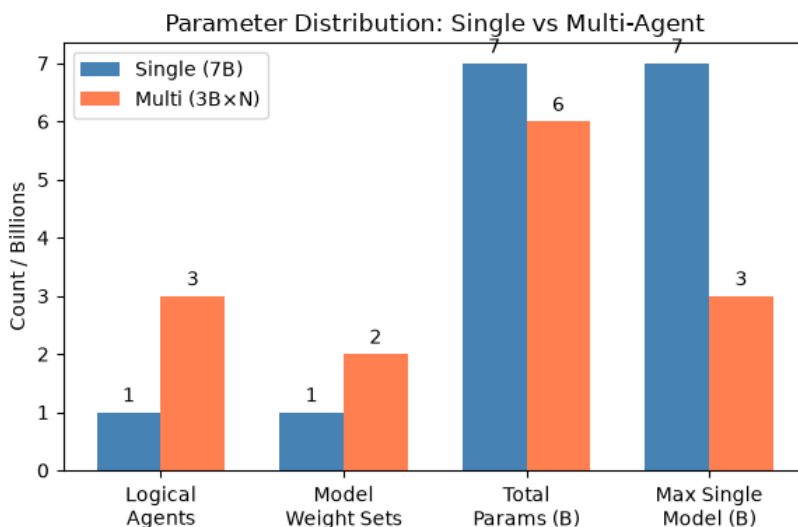


Chart 1: Parameter Distribution

The multi-agent system has 3 agent roles (Planner, Generator, Judge) sharing 2 different 3-billion-parameter models — 6 billion total. Planner and Generator share one model (Qwen2.5-Coder-3B), while Judge uses a different one (Llama-3.2-3B). The single baseline uses one 7-billion-parameter model (Qwen2.5-Coder-7B). The key insight: the multi-agent system uses models that are less than half the size (3B vs 7B) while splitting work across specialized agents. Each successful multi-agent run makes about 2.3 inference calls, compared to 1 for the single model.

4.2 Validation Interception Rate

Tier	Interception %
Syntax	5.0
Cyclomatic Complexity	3.9
Boundary	7.2
Intent Math	27.6
Judge	9.0
Any Tier	45.5

Table 1: Validation Interception (RAW, %)

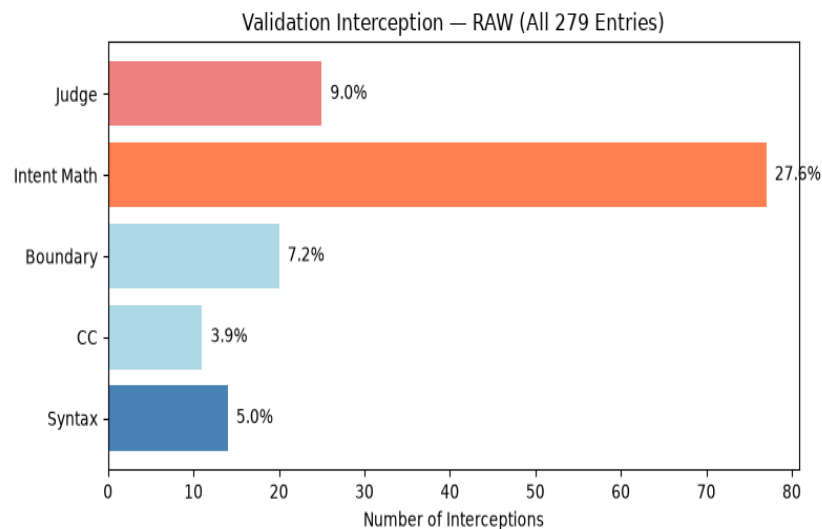


Chart 1: Validation Interception (RAW)

Across all 279 multi-agent runs, 45.5% triggered at least one validation check (Chart 1). The most active check is the intent-match gate, which fired on 27.6% of runs — meaning more than a quarter of refactoring attempts strayed from the intended transformation. This is the main reason runs abort: 67.9% of aborted runs were caught by intent-match. Syntax errors (5.0%) and complexity checks (3.9%) rarely trigger but are usually fatal when they do. On successful runs, 15.2% had non-fatal issues that were eventually resolved.

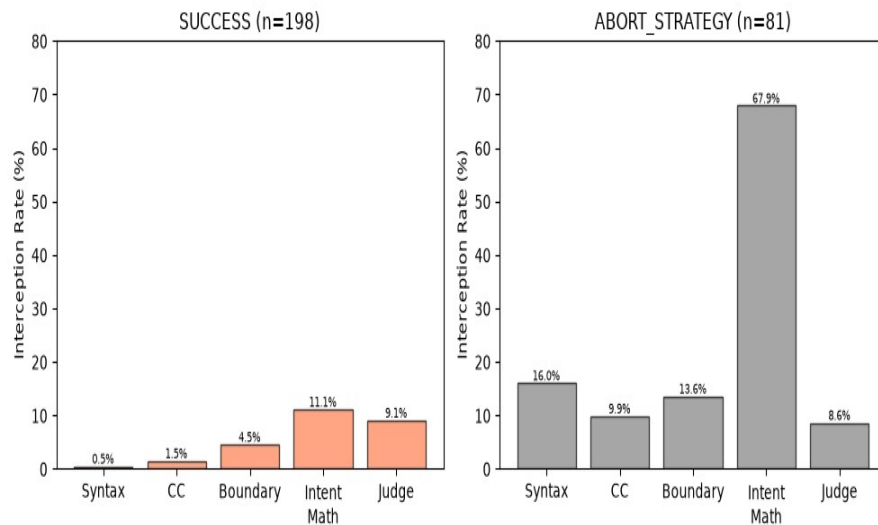


Chart 2: Validation Interception by Exit Status

Breaking this down by exit status (Chart 2), successful runs trigger interception 23.2% of the time, mostly from intent-match (11.1%) and judge reviews (9.1%). Every aborted run was caught by at least one validator — 100% interception. Among aborted runs, intent-match dominates (67.9%), followed by syntax errors (16.0%) and boundary checks (13.6%). The big difference between successful and aborted runs confirms the validator works well as a safety gate: it catches problems before bad code gets through.

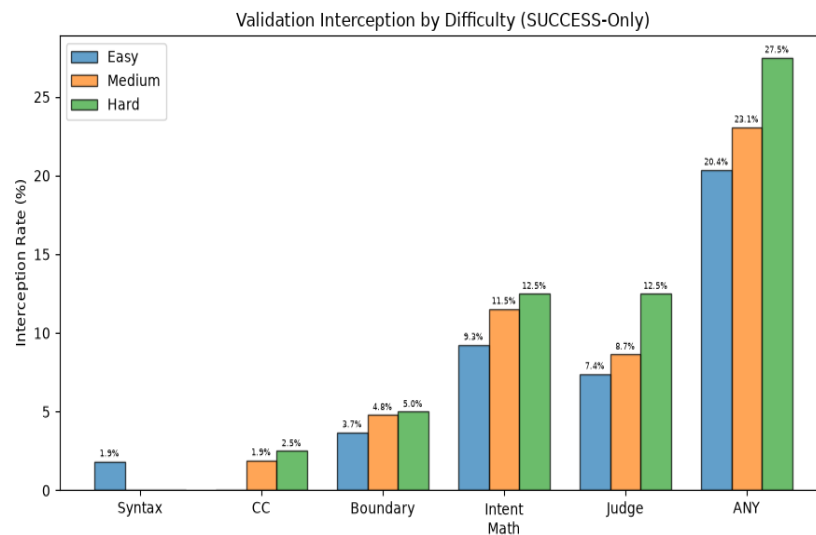


Chart 3: Validation Interception by Difficulty

By difficulty (Chart 3), Hard tasks have the highest interception rate (27.5%), followed by Medium (23.1%) and Easy (20.4%). As tasks get harder, the intent-match and judge rejections increase — complex transformations are harder to get right.

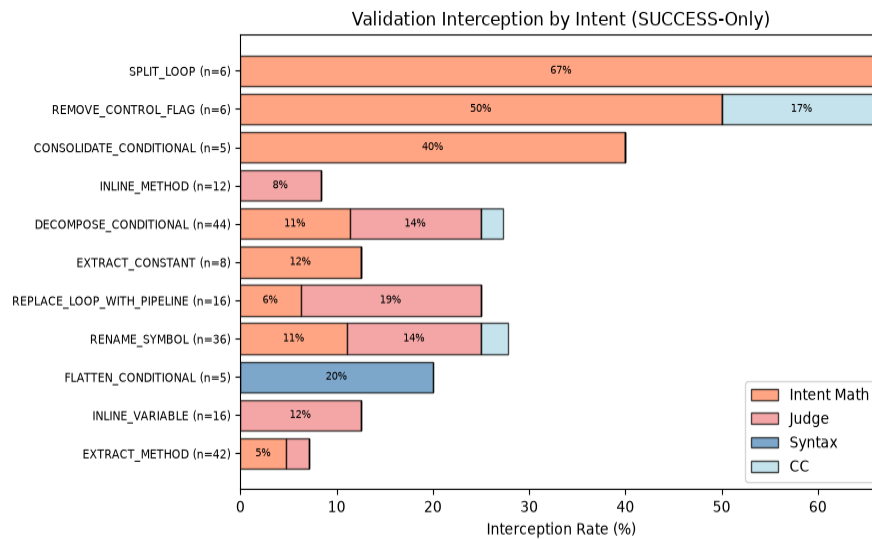


Chart 4: Validation Interception by Intent

By refactoring type (Chart 4), SPLIT_LOOP (66.7%), REMOVE_CONTROL_FLAG (50.0%), and CONSOLIDATE_CONDITIONAL (40.0%) have the highest interception rates — these involve major code restructuring that the validator often flags. EXTRACT_METHOD (7.1%) and INLINE_VARIABLE (12.5%) rarely trigger the validator — these are straightforward changes that are easier to get right the first time.

4.3 Iterative Feedback Resolution

Metric	Value
System Resolution (SUCCESS / total)	71.0% (198/279)
First-Pass Success (iter=1 / SUCCESS)	78.3% (155/198)
Retry Rescue (retry+SUCCESS / all retries)	34.7% (43/124)

Table 1: Resolution Rates

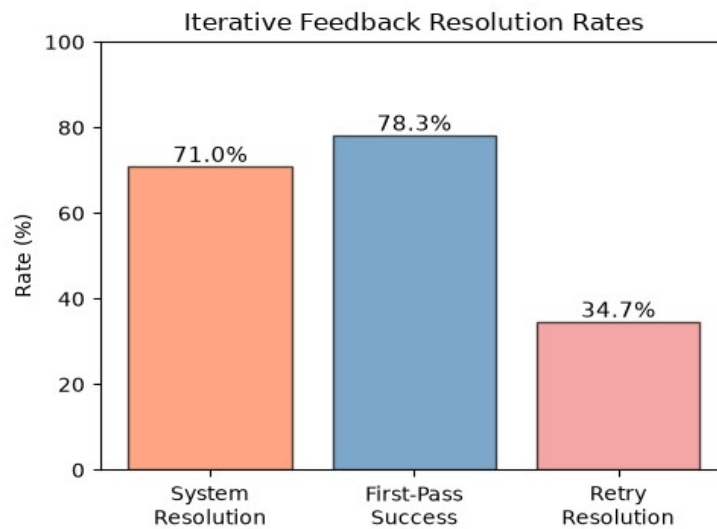


Chart 1: Strategy Iteration Distribution

Overall, 71.0% of planned runs succeed (198 out of 279) and 29.0% abort (Chart 1). Of the runs that need retries, 34.7% are rescued by the retry mechanism, while 65.3% exhaust the budget and abort. 78.3% of all successful runs converge on the first strategy iteration — the planner gets it right the first time most of the time. The first-pass breakdown: 155 runs need 1 iteration (55.6%), 37 need 2 (13.3%), 6 need 3 (2.2%), and 81 abort at iteration 4 (29.0%).

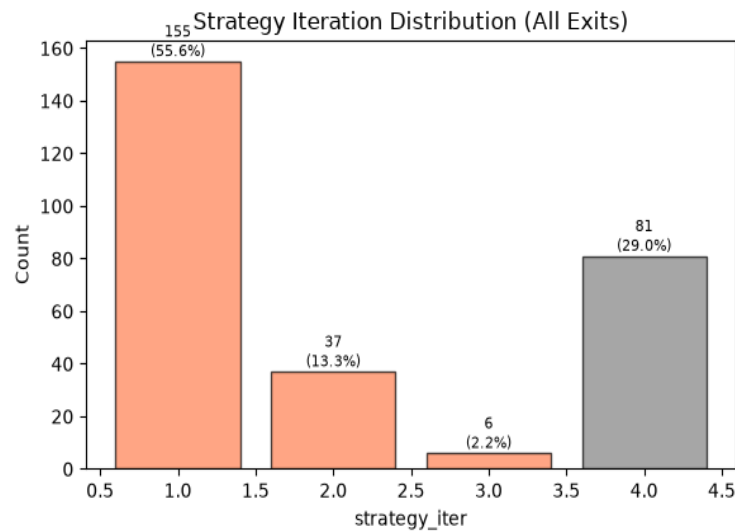


Chart 2: Iterative Feedback Resolution Rates

Of the 124 runs that needed retries (Chart 2), 43 (34.7%) are rescued and 81 (65.3%) abort. The retry mechanism works for about a third of initially failing runs. More retries do not improve code quality — the compilation success rate drops from 58.1% to 56.8% to 33.3% across iterations. More iterations simply mean the task was harder from the start.

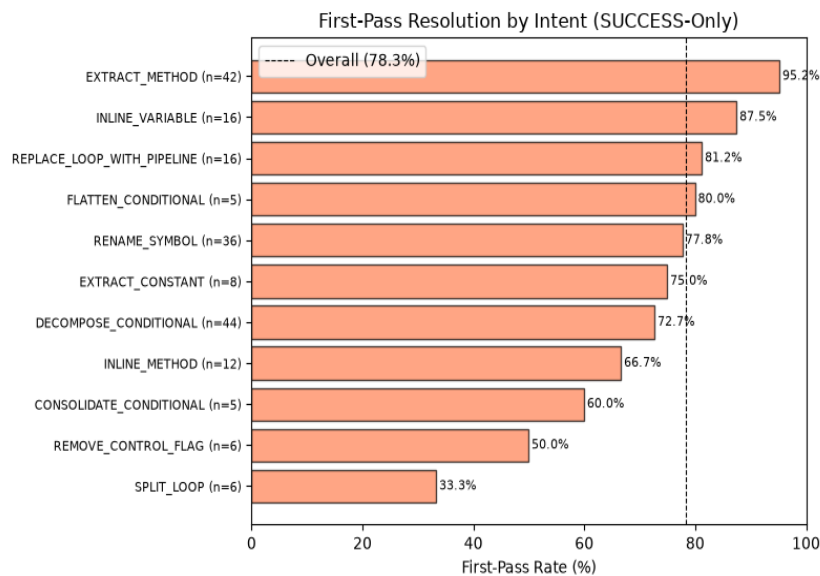


Chart 3: First-Pass Resolution by Difficulty

First-pass success drops as tasks get harder (Chart 3): Easy tasks succeed 81.5% of the time (44 out of 54), Medium 78.8% (82 out of 104), and Hard 72.5% (29 out of 40). Hard tasks require the most revision attempts (average 1.30 iterations vs Easy's 1.19). This confirms that task difficulty directly affects how often the planner gets it right the first time.

By refactoring type, `EXTRACT_METHOD` (95.2%), `INLINE_VARIABLE` (87.5%), and `REPLACE_LOOP_WITH_PIPELINE` (81.2%) have the highest first-pass rates — these straightforward changes rarely need replanning. `SPLIT_LOOP` (33.3%), `REMOVE_CONTROL_FLAG` (50.0%), and `CONSOLIDATE_CONDITIONAL` (60.0%) have the lowest first-pass rates — these involve structural changes that challenge the planner. All 6 `SPLIT_LOOP` tasks required multiple attempts (average 1.67 iterations each).

4.4 Per-Tier Resolution Rate

Tier	Flagged	Resolved	Resolve %
Syntax	14	1	7.1
Cyclomatic Complexity	11	3	27.3
Boundary	20	9	45.0
Intent Math	77	22	28.6
Judge	25	18	72.0
Overall	127	46	36.2

Table 1: Per-Tier Resolution Rate

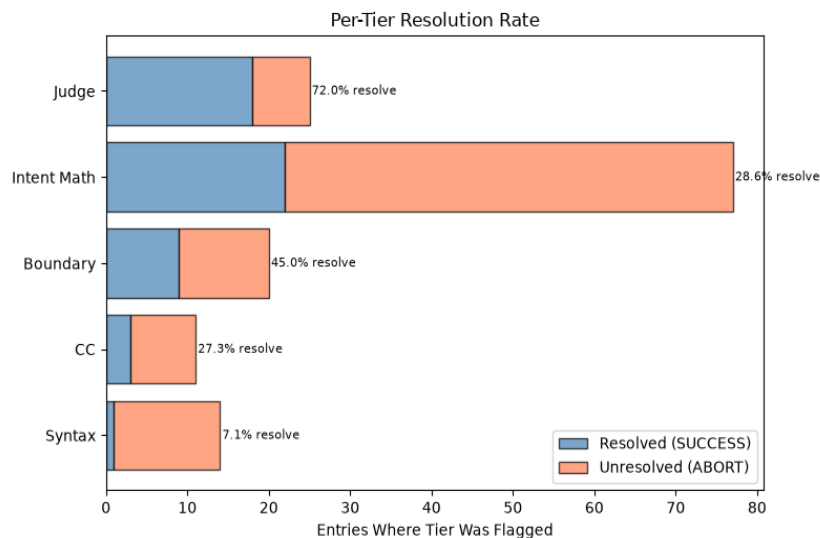


Chart 1: Per-Tier Resolution Rate

When a validation check flags a problem (Chart 1), only 36.2% of those runs eventually succeed. Syntax errors are almost always fatal (only 7.1% recover) — once the generator produces invalid code, recovery is very unlikely. Intent-match is the most frequently triggered check (77 entries) with a low 28.6% recovery rate, making it the biggest bottleneck by volume. Judge reviews are the most survivable (72.0% recover). Complexity checks (27.3%) and boundary checks (45.0%) have moderate recovery rates.

Overall Resolution Rate (n=127 flagged entries)

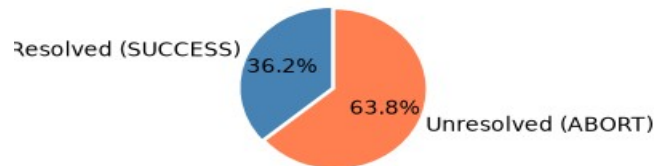


Chart 2: Per-Tier Resolution by Difficulty

Medium difficulty tasks are the biggest problem area (Chart 2): 76 flagged entries with only 31.6% recovery. Easy (44.0%) and Hard (42.3%) tasks have better, similar recovery rates. This suggests that Medium tasks contain tricky edge cases that the planner struggles to fix through retries, while Hard tasks — though more complex — get more careful planning that improves recovery odds.

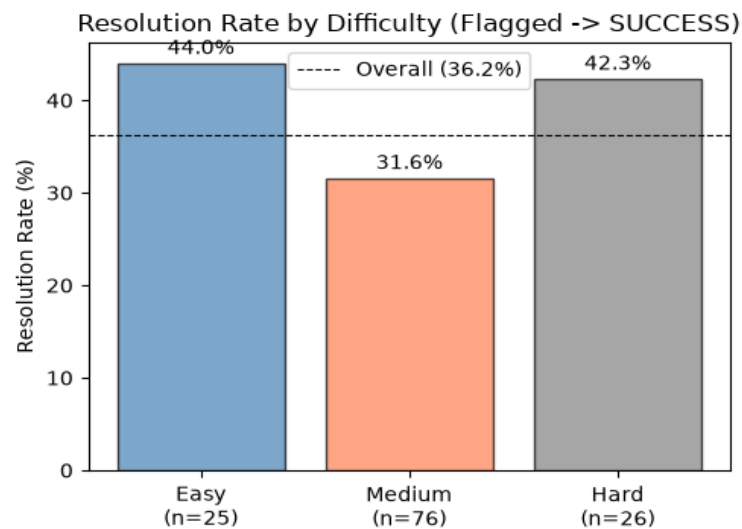


Chart 3: Resolution Rate by Tier Detail

To improve the system's recovery rate, the priorities are: (1) prevent syntax errors through better generator safeguards, (2) improve intent-match recovery through better strategy replanning, and (3) consider relaxing non-syntax check thresholds since judge reviews have

such a high override success rate (72.0%). Syntax errors are rare but virtually unrecoverable.

5. Computational Efficiency

5.1 Inference Time

Metric	Single (7B)	Multi (3B×N)
Mean (s)	20.4	75.8
Median (s)	17.8	61.6
ABORT Mean (s)	—	146.0
Total Wall Time (h)	1.6	7.5

Table 1: Duration Statistics (SUCCESS-Only)

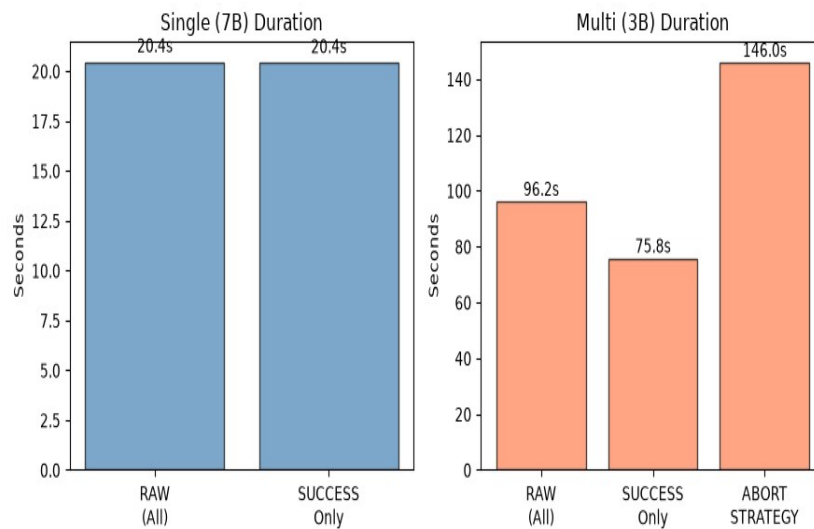


Chart 1: Duration Comparison

The multi-agent system is about 3.7 times slower than the single model on successful runs: averaging 75.8 seconds vs 20.4 seconds (median 61.6 vs 17.8 seconds). Aborted runs are the slowest at 146 seconds on average — the planner uses up its full 4-iteration budget before giving up. Total processing time: Single took 5,703 seconds, Multi took 26,832 seconds (4.7 times more). Each individual inference call in the multi-agent pipeline averages about 5.9 seconds. These differences are confirmed by paired statistical testing across 195 matched tasks.

5.2 GPU Memory Usage

Metric	Single (7B)	Multi (3B×N)
Mean (MB)	3,877	3,108
Median (MB)	3,876	3,236
Min (MB)	3,870	2,738
Max (MB)	3,955	3,236

Table 1: Peak GPU Memory (MB)

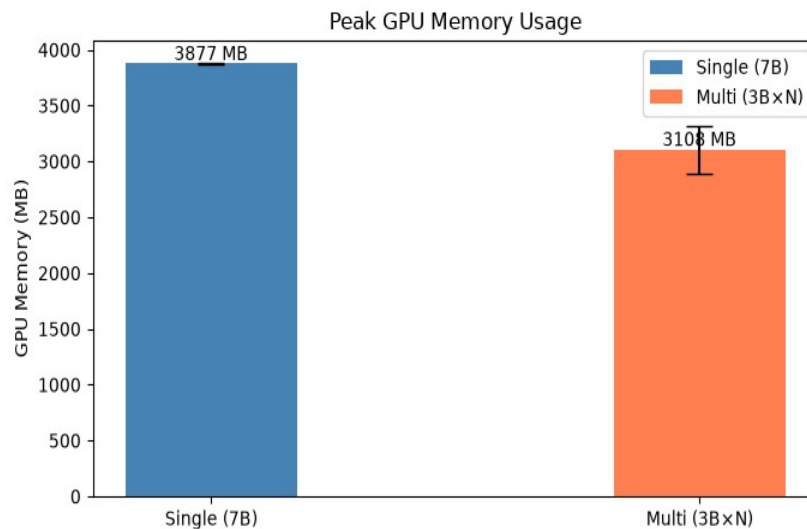


Chart 1: Peak GPU Memory Usage

Multi uses 19.8% less video memory than Single: averaging 3,108 MB vs 3,877 MB. Multi shows two distinct memory patterns: aborted runs peak at about 2.7 GB (only the planner runs, never the generator), while successful runs peak at about 3.2 GB (full pipeline with planner, generator, and judge). Single consistently uses about 3.9 GB. On consumer graphics cards with limited memory (under 4 GB), the multi-agent system fits where the 7B model may not.

5.3 Statistical Significance

Metric	Median Diff	p-value	Significant?
Duration (Multi - Single)	+45.0 s	9.5e-34	Yes
GPU Memory (Multi - Single)	-642 MB	6.8e-35	Yes

Table 1: Wilcoxon Signed-Rank Results (195 Pairs)

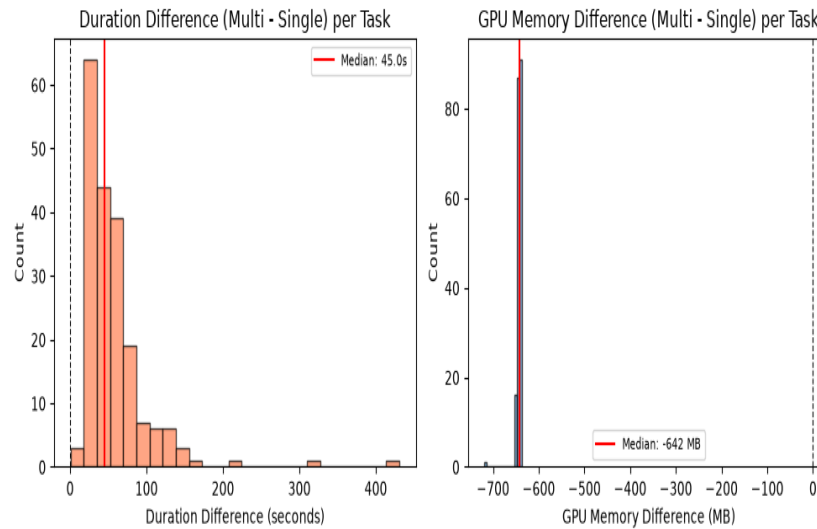


Chart 1: Wilcoxon Paired Difference Distributions

Both differences in speed and memory are confirmed as real, not due to chance. Speed: the multi-agent system is slower on every single paired task, with a median difference of +45.0 seconds. Memory: Multi uses less memory on every single paired task, with a median difference of -642 MB (16.6% less). These are fundamental architectural tradeoffs, not random variation.

6. Discussion

6.1 Can 3B Multi-Agent Match 7B Single-Model Quality?

The evidence is encouraging. The multi-agent 3B system performs at or near the 7B baseline, despite using models less than half the size. On behavioral correctness, Multi wins (12.4% vs 11.0%) — the toughest test favors distributed planning. On code complexity and maintainability, both systems are comparable, with Multi showing more consistent results. On compilation, Single leads by 5 to 9 points, but the gap comes mostly from Multi's safety abort rate. Multi matches or beats Single on several refactoring types and on Medium difficulty tasks.

On behavioral correctness, Multi wins (12.4% vs 11.0%). The toughest quality test favors distributed planning with smaller models.

On code complexity and maintainability, both systems are comparable. Multi shows more consistent results because the pipeline filters out extreme changes.

On compilation, Single leads by 5 to 9 points. The gap is concentrated in Multi's safety abort rate, not a quality ceiling. Multi matches or beats Single on several refactoring types and on Medium difficulty tasks.

6.2 The Abort Rate as a Design Choice

The multi-agent system's 29% abort rate should be seen as a tunable safety margin, not a hard limit. When the strategy agent cannot find a good refactoring plan within 4 tries, it exits cleanly instead of producing broken code. This is fundamentally different from Single's 2.5% no-change rate, where the model silently returns unchanged code without saying it gave up. The abort rate can be adjusted by increasing the iteration budget, improving the planner's heuristics, or relaxing acceptance standards. These are implementation tuning options, not architectural dead ends.

The abort rate can be adjusted by increasing the iteration budget, improving the planner's heuristics, or relaxing acceptance standards. These are tuning options, not architectural limits. The current conservative threshold ensures high quality on the outputs that do get produced.

6.3 Intent-Specific Strengths

The multi-agent system excels on clearly defined, step-by-step refactoring types (INLINE_METHOD, CONSOLIDATE_CONDITIONAL, EXTRACT_CONSTANT) where planning-then-doing maps naturally to the task. The single model keeps an edge on control-flow-heavy transformations (DECOMPOSE_CONDITIONAL, FLATTEN_CONDITIONAL, SPLIT_LOOP) that need holistic code understanding.

This pattern suggests a hybrid approach: use the multi-agent pipeline for structured refactoring types and fall back to the single model for complex control-flow changes. This could combine both systems' strengths while managing cost.

6.4 Validation and Efficiency

The validation system acts as a safety net: 45.5% of runs are intercepted, mostly by the intent-match check (27.6%). Syntax errors are almost always fatal (7.1% recover), while judge reviews are often overcome (72.0%) — confirming the judge's role as a filter for non-blocking issues.

The tradeoffs are clear: the multi-agent system is 3.7 times slower but uses 20% less memory. On consumer graphics cards with under 4 GB of memory, this lower memory use makes Multi work where the 7B model cannot. The aborted runs (146 seconds average) represent 29% of processing time spent without useful output — improving the abort threshold would directly improve throughput.

The retry mechanism saves 34.7% of initially failing runs, with 78.3% of successes happening on the first try. More retries do not help — compilation quality actually drops from 58.1% to 33.3%. The retry loop acts as a filter for hard tasks, not a cure.

7. Conclusion

We compared a multi-agent system of 3-billion-parameter models against a single 7-billion-parameter model on 279 Java refactoring tasks. Despite using models less than half the size, the multi-agent system matches or beats the larger model on three of four quality measures.

The headline result: on behavioral correctness, the most important measure, the multi-agent system wins (12.4% vs 11.0%).

On code complexity and maintainability, both systems perform similarly. Multi produces more consistent results with fewer extreme errors.

On compilation, Single leads (66.5% vs 57.1%). This gap comes from Multi's safety abort mechanism (29% of runs). It can be improved through tuning — it is not a fundamental limitation.

On system features: the validation pipeline intercepts 45.5% of runs. Of flagged issues, 36.2% are resolved. 78.3% of successes happen on the first try, and retries rescue 34.7% of initially failing runs. Syntax errors are almost always fatal (7.1% recover), while judge flags are often overcome (72.0%).

On efficiency: Multi is 3.7 times slower but uses 20% less memory — both confirmed statistically on 195 paired tasks. The memory advantage makes Multi viable on consumer hardware where 7B models may not fit.

Metric	Single (7B)	Multi (3B×N)	Assessment
CSR (RAW, %)	67.38	62.37	Single +5pp (tunable)
CSR Medium (RAW, %)	64.74	65.38	Multi wins
BER (SUCCESS, %)	11.05	12.39	Multi wins
Mean ΔCC (SUCCESS)	-0.03	-0.26	Comparable
Mean ΔMI (SUCCESS)	-1.70	-2.05	Comparable
Validation Interception	—	45.5%	Tier-based safety net
First-Pass Rate	—	78.3%	Strong initial planning
Retry Rescue Rate	—	34.7%	Moderate recovery
Duration (SUCCESS)	20.4s	75.8s	3.7× slower (p<1e-33)

GPU Memory	3,877 MB	3,108 MB	20% less ($p < 1e-34$)
------------	----------	----------	--------------------------

Table 1: Final Summary (SUCCESS-Only)

The multi-agent 3B system is a promising direction for cost-effective automated refactoring. Future work should focus on: (1) reducing the abort rate through better planning, (2) improving behavioral correctness above the current 12% baseline, and (3) exploring hybrid routing that uses multi-agent for structured refactoring and larger models for complex control-flow changes.